

Generative AI and RAG

Expanded Knowledge Base Document for RAG / Document Question-Answering

1. Generative AI

Generative artificial intelligence refers to systems that create new content such as text, images, audio, or code from learned patterns. Large language models are generative models that can produce text conditioned on a prompt and conversation context. Their outputs depend on both the learned model and the information supplied during interaction.

2. Large Language Models

Large language models process sequences of tokens and learn statistical relationships from large collections of text. They can be used for tasks such as question answering, summarization, rewriting, extraction, and conversational assistance. A prompt provides instructions or context that helps define the desired output.

3. Retrieval-Augmented Generation

Retrieval-augmented generation, commonly called RAG, combines information retrieval with language generation. A collection of documents is divided into chunks, each chunk is represented by an embedding, and the embeddings are stored in a vector database. When a user asks a question, the question is embedded and compared with stored embeddings.

4. RAG Pipeline

A typical RAG pipeline has document ingestion, parsing, chunking, embedding generation, vector storage, retrieval, prompt construction, and answer generation. During ingestion, documents are loaded and converted into usable text. Chunking divides long documents into smaller units. Embeddings represent semantic information, while retrieval selects chunks relevant to a query. The generator receives the retrieved material as context.

5. Chunking

Chunk size affects retrieval quality. Very small chunks may lose context, while very large chunks can contain unrelated material and consume more context space. Overlap between adjacent chunks can preserve information that crosses chunk boundaries. The best settings depend on the document structure and the questions the application must answer.

6. Embeddings and Vector Search

An embedding converts text into a numerical representation. Semantically related text can have similar representations, allowing retrieval systems to compare a query with document chunks. A vector database or another vector index stores these representations and supports similarity-based retrieval. Metadata can also be stored so that retrieved content can be traced back to its source document.

7. Generation and Grounding

After retrieval, the selected chunks are supplied to a language model as context. The model then generates an answer based on that retrieved material. A useful RAG prompt can instruct the model to rely on the supplied context and to indicate when the documents do not contain enough information. Showing source metadata can make answers easier to verify.

8. RAG Quality Factors

RAG quality depends on document parsing, chunk size, embedding quality, retrieval settings, and the generation prompt. Poor parsing can remove useful text. Weak embeddings can reduce retrieval relevance. Incorrect retrieval settings can return too little or too much context. Even when retrieval is correct, an unclear generation prompt can produce an incomplete response.

9. RAG Versus Retraining

RAG can help a language model work with private, recent, or specialized documents without retraining the model. Updating the knowledge base can therefore be simpler than changing the model itself. RAG is especially useful when the application needs answers grounded in a controlled collection of documents.

10. Example Document QA System

A document question-answering application can store several PDFs in a data folder, extract their text, split the text into chunks, create embeddings, and save them in a vector index. For a user question, the system retrieves relevant chunks and passes them to the language model. The interface can display the answer together with document names or other source metadata.

11. Key Takeaways

RAG is a pipeline that connects document retrieval with language generation. Its usefulness depends on every stage, from document preparation to retrieval and final generation. Good source management and traceability are important for building a reliable document question-answering system.